

Gradient Boosting - Algorithm (Regression)

Toy-dataset

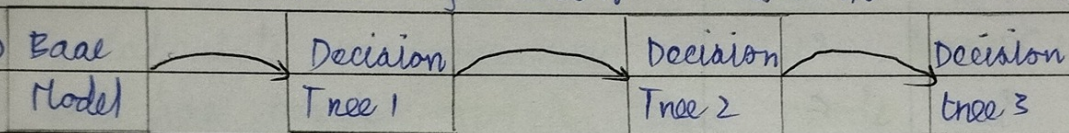
id	cgpa	salary
90	8	3
100	7	4
110	6	8
120	9	6
80	5	3

Gradient-Boosting is a machine learning algorithm that builds a strong predictive model by combining many **weak learners** (**models**) usually small decision trees sequentially

$$3 + 4 + 8 + 6 + 3 / 5 = 4.8$$

Base Model in Regression :- ~~output~~ ^{PUT} column **Mean**

this is called as DT leaf



⇒ This is called as **sequentially stage-wise addition**

The Gradient Boosting way

↳ In gradient boosting, sequential stage-wise learning means :-

"Build your model by adding one tree at a time, and never change the trees you've already built".

⇒ Why "sequentially" ?

↳ Trees are built **one after another**, not simultaneously. Tree₂ can't be built until Tree₁ is done because Tree₂ needs to see Tree₁'s mistakes.

⇒ Why "Stage-wise" ?

↳ At each stage, we only add something new. We **never go back** and retrain or modify earlier trees. They're locked in place.

→ In Gradient Boosting the first model is called as **Base Model** (Loat) which is nothing but **mean** of output column (dependent variables)

→ For eg :- $3 + 4 + 8 + 6 + 3 = 4.8$
5

Dataset

iq	cgpa	salary
90	8	3
100	7	4
110	6	8
120	9	6
80	5	3

* The base model (initial model) predicts the same mean value for every single sample whether its training data or test data

- Base Model learns: mean = **4.8** (from training data)
- Applied to any new test sample: predicts **4.8**
- Applied to another test sample :- predicts **4.8**
- Applied to another test sample :- predicts **4.8**

⇒ The base model is "**dump**" - it just outputs that one constant value (the **mean**) regardless of the input features.

Step 1:- Calculate Base Model Prediction (Output)

iq	cgpa	salary	pred 1	∴ pred 1 is nothing but prediction from base model based on input and output data is always mean of output column which is = 4.8 always.
90	8	3	4.8	
100	7	4	4.8	
110	6	8	4.8	
120	9	6	4.8	
80	5	3	4.8	

Step 2:- Calculate the performance of the base model

→ To calculate performance we use a **Loss (cost) function** and how much **mistakes** they made and we tell this **base model mistakes** to second model (**Decision tree**)

⇒ (actual - predicted value) ⇒ $(y - \hat{y})$

↳ We use this **log loss function** and in gradient Boosting it is called as **pseudo-residuals**

iq	cgpa	salary	pred 1	res 1 (M)
90	8	3	4.8	$3 - 4.8 = -1.8$
100	7	4	4.8	$4 - 4.8 = -0.8$
110	6	8	4.8	$8 - 4.8 = 3.2$
120	9	6	4.8	$6 - 4.8 = 1.2$
80	5	3	4.8	$3 - 4.8 = -1.8$

• **Step 3:** Send the base model's errors to second model which is our (decision - tree)

- Model 1 :- base Model (Mean) (r1) ⇒ residuals

- Model 2 :- Decision - tree

in step 3 we send this errors to Model 2

Now Question is ?

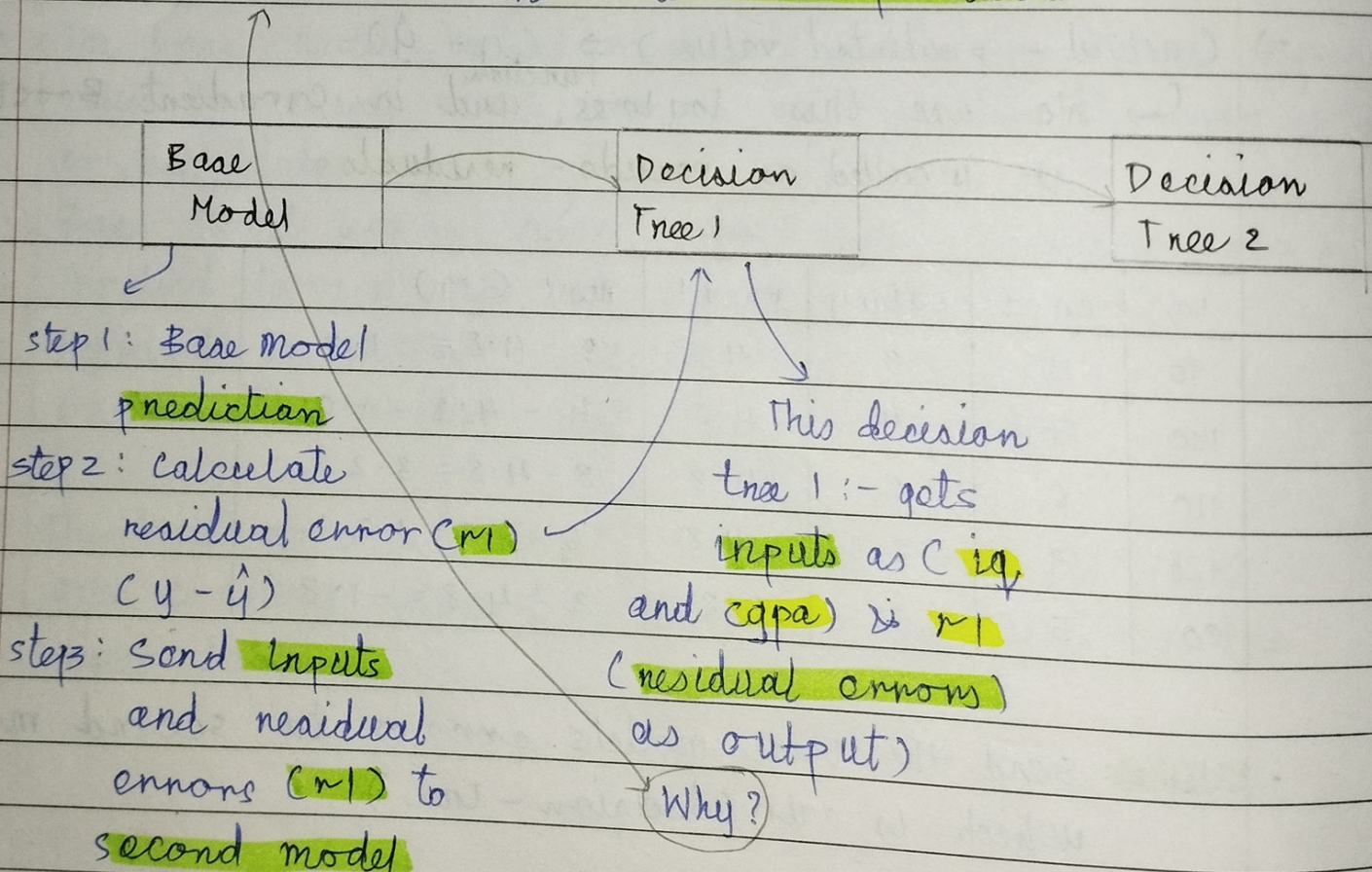
How we train the decision - tree

Solution :- We give Independent features as **Inputs** which is iq and cgpa in our case

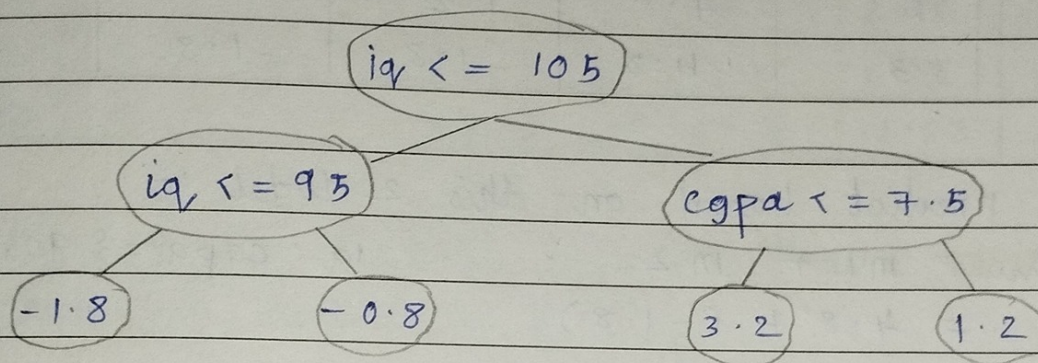
For output we give **residual error (r1)** of base model not salary (independent feature or actual output value)

Now Question is Why Residuals as output (not original y) :

We give residuals as target (not y) because instead of predicting the original output, we want to predict **how much error the base model produced**.



iq	cgpa	salary	pred 1	res1 (r1)	pred 2
90	8	3	4.8	-1.8	-1.8
100	7	4	4.8	-0.8	-0.8
110	6	8	4.8	3.2	3.2
120	9	6	4.8	1.2	1.2
80	5	3	4.8	-1.8	-1.8



? How 105 is chosen

↳ The algorithm tests ALL possible split points between consecutive IQ values:

— Your IQ values sorted: 80, 90, 100, 110, 120

• Possible split points to test :-

- Between 80 and 90 → test split at 85
- Between 90 and 100 → test split at 95
- Between 100 and 110 → test split at 105 ✓
- Between 110 and 120 → test split at 115

Why 105 wins :-

From my calculations above:

- Split at 85 : MSE = 2.950
- Split at 95 : MSE = 1.600
- Split at 105 : MSE = 0.533 ← Lowest (Winner)
- Split at 115 :- MSE = 3.400

⇒ same process for 95 and CGPA 7.5

⇒ This is our data from Model 1 and Model 2

iq	cgpa	salary	pred 1	res 1	pred 2
90	8	3	4.8	-1.8	-1.8
100	7	4	4.8	-0.8	-0.8
110	6	8	4.8	3.2	3.2
120	9	6	4.8	1.2	1.2
80	5	3	4.8	-1.8	-1.8

⇒ Now let's predict based on this 2 Model :-

$$\begin{aligned}
 \hookrightarrow \text{prediction} &= m_1 + m_2 & \text{iq: cgpa} &= \{90, 8\} \\
 &= 4.8 + (-1.8) \\
 &= 4.8 - 1.8 \\
 &= 3 \Rightarrow \text{Overfitting}
 \end{aligned}$$

↳ To prevent model from overfitting we use α :- (learning rate)

New formula :-

$$\begin{aligned}
 \Rightarrow \text{prediction} &= m_1 + \alpha m_2 & 0.1 \text{ (can be any value} \\
 & & \text{between 0 \& 1)} \\
 &= 4.8 + 0.1(-1.8) \\
 &= 4.62
 \end{aligned}$$

$$\Rightarrow \text{Formula :- } \hat{y}_M(x) = \hat{y}_0 + \sum_{t=1}^m \alpha \cdot h_m(x)$$

IMP:-

⇒ To generalize better we use a regularization technique called as **shrinkage** ⇒ We set low learning rate (α) value such as 0.05 and more trees like 48, 53, 92 etc depending upon volume of datasets.

Step 4:- Calculate performance of Model 2 (residual errors)
 $(y - \hat{y})$

iq	cgpa	salary	pred 1	res 1	pred 2	res 2
90	8	3	4.8	-1.8	-1.8	-1.62
100	7	4	4.8	-0.8	-0.8	-0.72
110	6	8	4.8	3.2	3.2	2.88
120	9	6	4.8	1.2	1.2	1.08
80	5	3	4.8	-1.8	-1.8	-1.62

- The Formula used to calculate Model 2 residual error
 (same for Model 3, 4, 5 n)

~~$$\hat{y}_M(x) = \hat{y}_0 + \sum_{i=1}^M v_i \cdot h_m(x)$$~~

$$\Rightarrow r_M(x) = y(x) - \left(\hat{y}_0 + \sum_{i=1}^M v_i \cdot h_m(x) \right)$$

$\xrightarrow{\text{learning rate}}$
 $\xrightarrow{\text{predicted value of (Decision Tree) pred 2}}$
 $\xrightarrow{\text{predicted value (base model) pred 1}}$
 $\xrightarrow{\text{actual value (salary)}}$

$\Rightarrow$ We can see that error has reduced from error 1 (res 1) to error 2 (res 2) $\Rightarrow$ when it becomes 0 or very near to 0 it predicts the actual value

$\Rightarrow$ we need residual errors to be 0 in order to predict actual value and from above dataset we can see that adding more decision trees, residual errors comes near to 0.

$\Rightarrow$ Therefore, we need to add more decision trees to reduce errors and bring them to 0.

Step 3 :- calculate another decision-tree to bring residual errors near to 0 or exactly 0.

iq	cgpa	salary	pred 1	res 1	pred 2	res 2	pred 3	res 3
90	8	3	4.8	-1.8	-1.8	-1.62	-1.62	-1.45
100	7	4	4.8	-0.8	-0.8	-0.72	-0.72	-0.64
110	6	8	4.8	3.2	3.2	2.88	2.88	2.59
120	9	6	4.8	1.2	1.2	1.08	1.08	0.972
80	5	3	4.8	-1.8	-1.8	-1.62	-1.62	-1.45

